Ejercicios de JOINs

Tabla informativa sobre SELECT

Funcionalidad	¿Para qué sirve?	Ejemplo
ORDER BY	Ordena los resultados de una consulta de forma ascendente o descendente.	SELECT * FROM Books ORDER BY Name ASC;
LIMIT	Limita la cantidad de filas que devuelve una consulta.	SELECT * FROM Books LIMIT 3;
GROUP BY	Agrupa filas que tienen valores iguales. Se usa comúnmente con funciones como COUNT, SUM o AVG.	SELECT Author, COUNT(*) FROM Books GROUP BY Author;
INNER JOIN	Devuelve solo los registros que tienen coincidencias en ambas tablas.	SELECT * FROM Books INNER JOIN Authors ON Books.Author = Authors.ID;
LEFT JOIN	Devuelve todos los registros de la tabla izquierda y las coincidencias de la tabla derecha. Si no hay coincidencia, muestra NULL.	SELECT * FROM Books LEFT JOIN Authors ON Books.Author = Authors.ID;
CROSS JOIN	Combina cada fila de una tabla con cada fila de otra tabla.	SELECT * FROM Books CROSS JOIN Authors;

Creación de tablas

```
CREATE TABLE Authors (
    ID INTEGER PRIMARY KEY,
    Name TEXT NOT NULL
);

CREATE TABLE Books (
    ID INTEGER PRIMARY KEY,
    Name TEXT NOT NULL,
    Author INTEGER,
    FOREIGN KEY (Author) REFERENCES Authors(ID)
);

CREATE TABLE Customers (
    ID INTEGER PRIMARY KEY,
    Name TEXT NOT NULL,
```

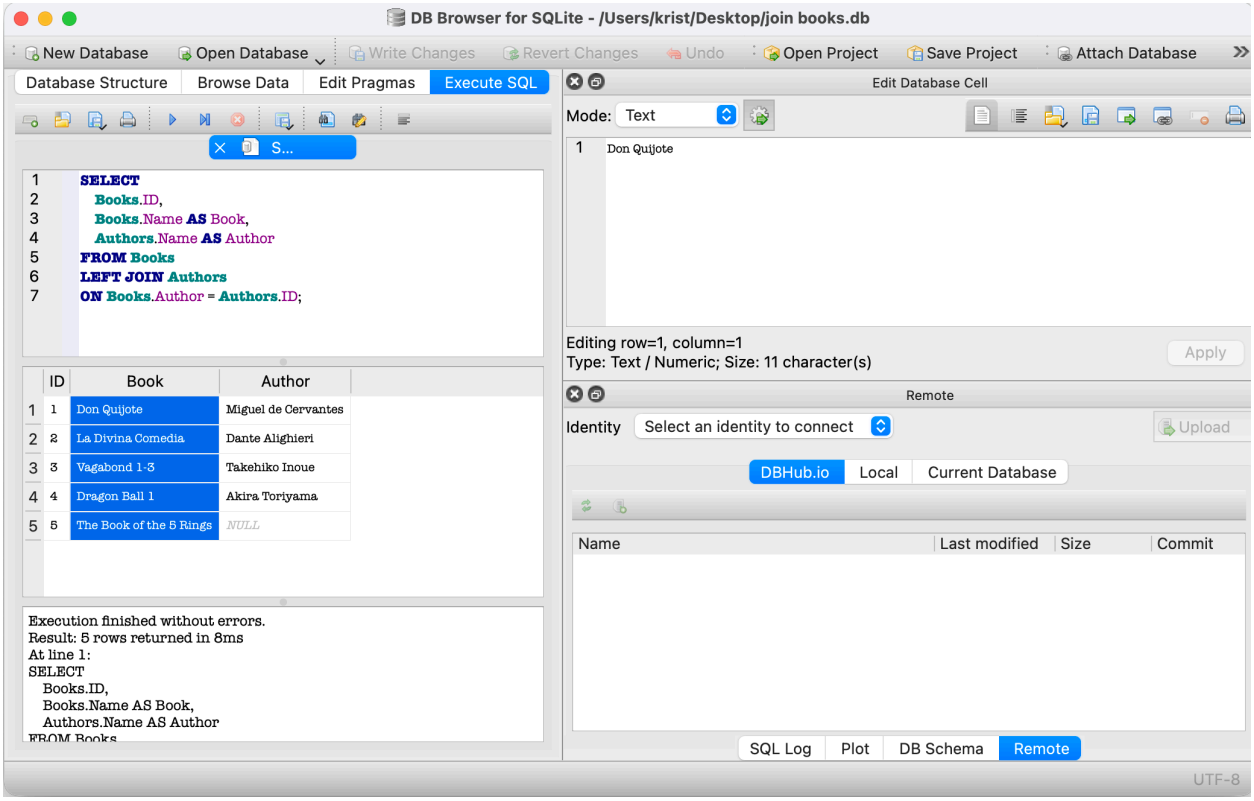
```
    Email TEXT NOT NULL
);

CREATE TABLE Rents (
    ID INTEGER PRIMARY KEY,
    BookID INTEGER,
    CustomerID INTEGER,
    State TEXT NOT NULL,
    FOREIGN KEY (BookID) REFERENCES Books(ID),
    FOREIGN KEY (CustomerID) REFERENCES Customers(ID)
);
```

Inserción de datos

```
INSERT INTO Authors (ID, Name) VALUES
(1, 'Miguel de Cervantes'),
(2, 'Dante Alighieri'),
(3, 'Takehiko Inoue'),
(4, 'Akira Toriyama'),
(5, 'Walt Disney');

INSERT INTO Books (ID, Name, Author) VALUES
(1, 'Don Quijote', 1),
(2, 'La Divina Comedia', 2),
(3, 'Vagabond 1-3', 3),
(4, 'Dragon Ball 1', 4),
(5, 'The Book of the 5 Rings', NULL);

INSERT INTO Customers (ID, Name, Email) VALUES
(1, 'John Doe', 'j.doe@email.com'),
(2, 'Jane Doe', 'jane@doe.com'),
(3, 'Luke Skywalker', 'darth.son@email.com');

INSERT INTO Rents (ID, BookID, CustomerID, State) VALUES
(1, 1, 2, 'Returned'),
(2, 2, 2, 'Returned'),
(3, 1, 1, 'On time'),
(4, 3, 1, 'On time'),
(5, 2, 2, 'Overdue');
```

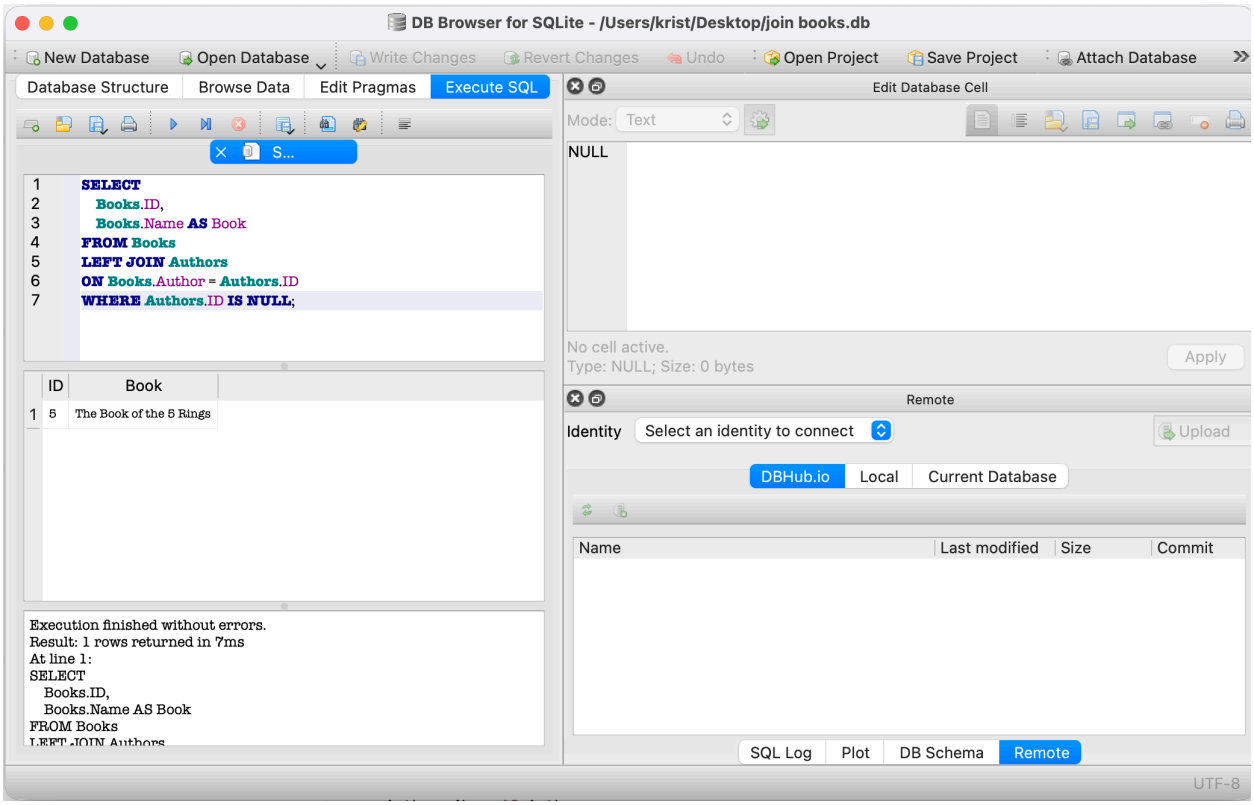
1. Obtener todos los libros y sus autores, en caso de tenerlos

```
SELECT
    Books.ID,
    Books.Name AS Book,
    Authors.Name AS Author
FROM Books
LEFT JOIN Authors
ON Books.Author = Authors.ID;
```



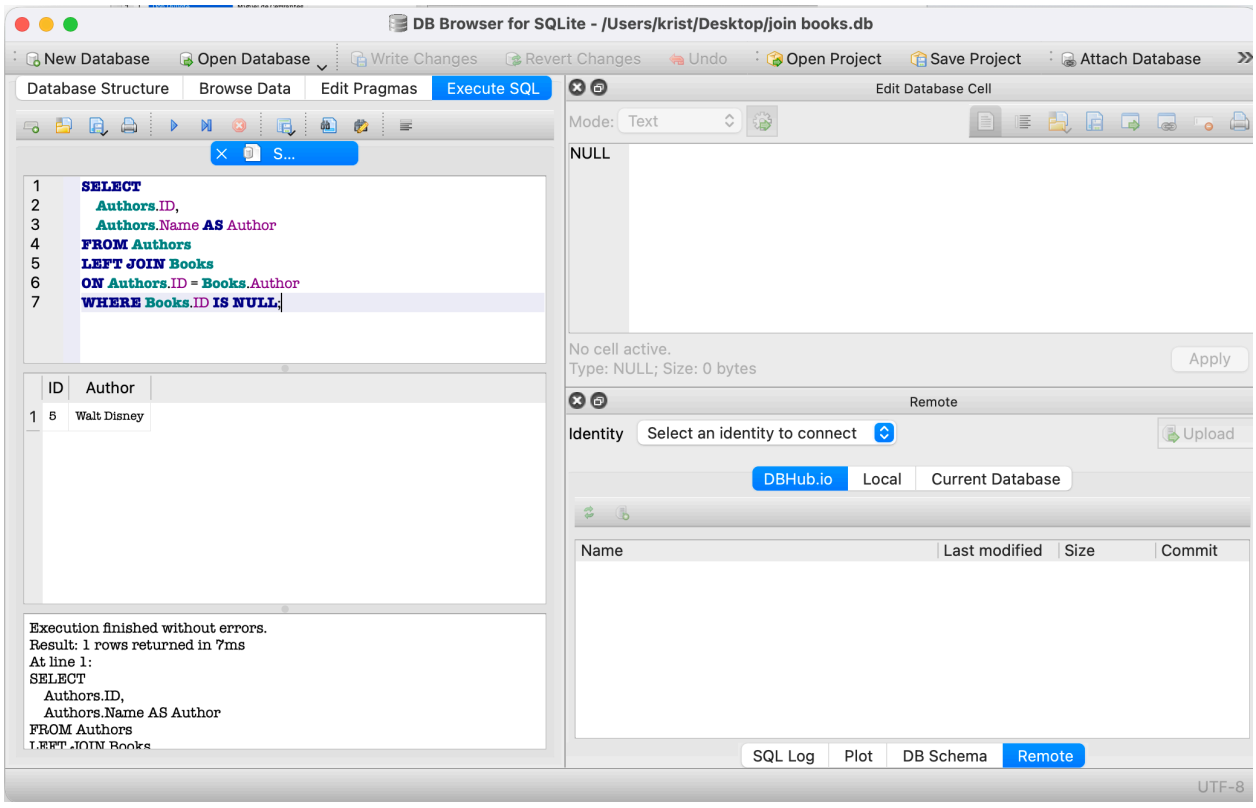
2. Obtener todos los libros que no tienen autor

```
SELECT
    Books.ID,
    Books.Name AS Book
FROM Books
LEFT JOIN Authors
ON Books.Author = Authors.ID
WHERE Authors.ID IS NULL;
```



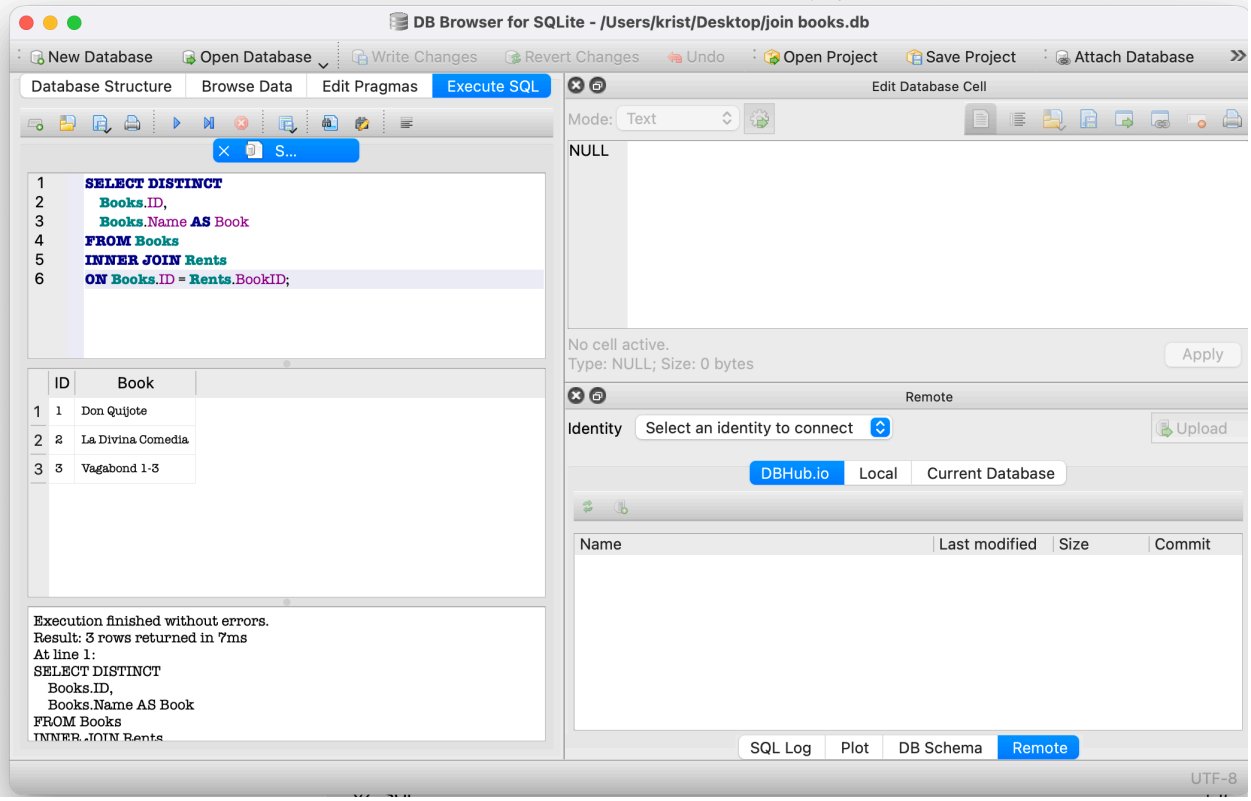
3. Obtener todos los autores que no tienen libros

```
SELECT
    Authors.ID,
    Authors.Name AS Author
FROM Authors
LEFT JOIN Books
ON Authors.ID = Books.Author
WHERE Books.ID IS NULL;
```



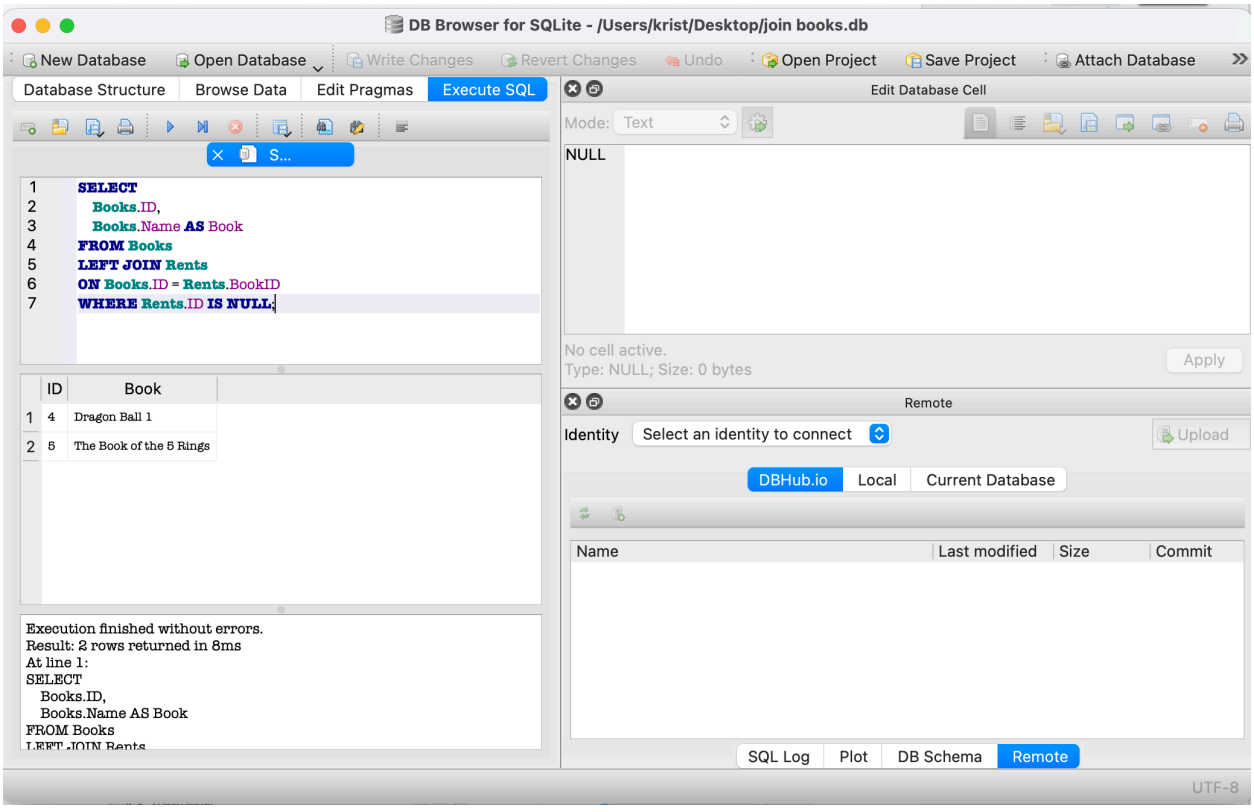
4. Obtener todos los libros que han sido rentados en algún momento

```
SELECT DISTINCT
    Books.ID,
    Books.Name AS Book
FROM Books
INNER JOIN Rents
ON Books.ID = Rents.BookID;
```



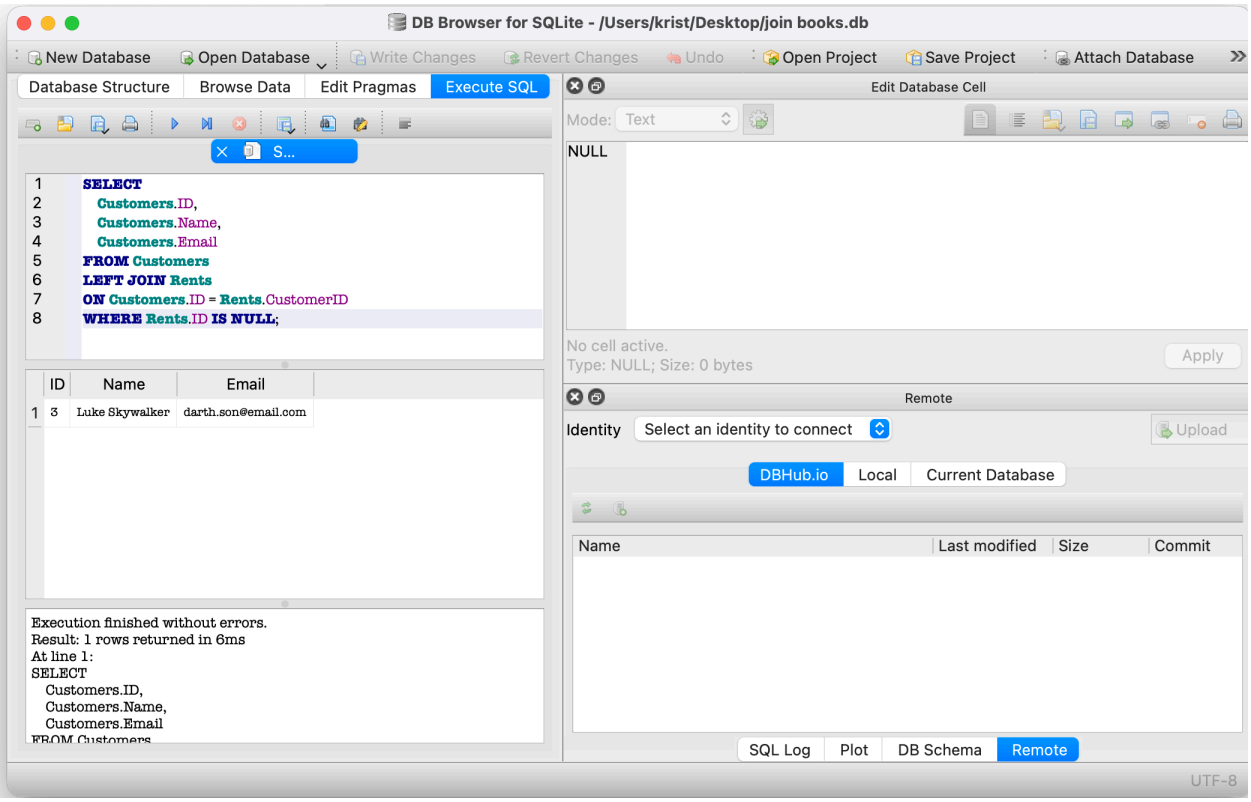
5. Obtener todos los libros que nunca han sido rentados

```
SELECT
    Books.ID,
    Books.Name AS Book
FROM Books
LEFT JOIN Rents
ON Books.ID = Rents.BookID
WHERE Rents.ID IS NULL;
```



6. Obtener todos los clientes que nunca han rentado un libro

```
SELECT
    Customers.ID,
    Customers.Name,
    Customers.Email
FROM Customers
LEFT JOIN Rents
ON Customers.ID = Rents.CustomerID
WHERE Rents.ID IS NULL;
```



7. Obtener todos los libros que han sido rentados y están en estado “Overdue”

```
SELECT
    Books.ID,
    Books.Name AS Book,
    Rents.State
FROM Books
INNER JOIN Rents
ON Books.ID = Rents.BookID
WHERE Rents.State = 'Overdue';
```